



Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

Threads and Interprocess Communication

Rehan Naeem

University of Engineering and Technology Lahore

September 2025



Outline

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

1 Fork Join Concept

2 Creating Threads in A Class

3 Dynamic Threads

4 Events and Mailbox

5 Unsynchronized Thread Communication

6 Synchronized Thread Communication



Fork Join

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

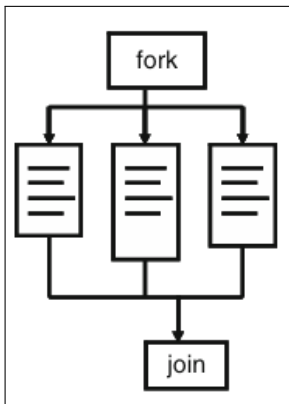
Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication





Fork Join

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join
    $display("@%0t: after join", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

(a) Code



Fork Join

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join
    $display("@%0t: after join", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

(a) Code

```
@0: start fork...join example
@10: sequential after #10
@10: parallel start
@20: parallel after #10
@40: sequential after #30
@50: sequential after #10
@60: parallel after #50
@60: after join
@140: finish after #80
```

(b) Output



Fork Join_none

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

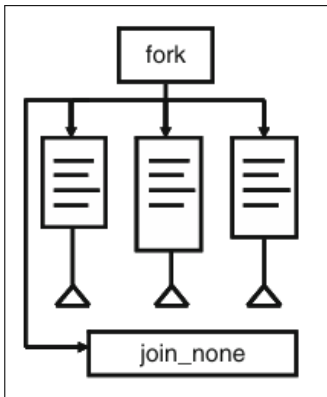
Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication





Fork Join_none

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsyncronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join_none example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join_none
        $display("@%0t: after join_none", $time);
        #80 $display("@%0t: finish after #80", $time);
    end
end
```

(a) Code



Fork Join_none

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join_none example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join_none
    $display("@%0t: after join_none", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

(a) Code

```
@0: start fork...join_none example
@10: sequential after #10
@10: after join_none
@10: parallel start
@20: parallel after #10
@40: sequential after #30
@50: sequential after #10
@60: parallel after #50
@90: finish after #80
```

(b) Output



Fork Join_any

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

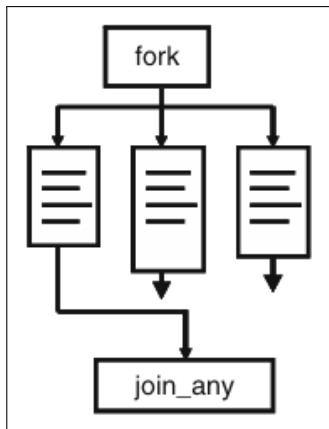
Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication





Fork Join_any

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join_any example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join_any
    $display("@%0t: after join_any", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

(a) Code



Fork Join_any

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
initial begin
    $display("@%0t: start fork...join_any example", $time);
    #10 $display("@%0t: sequential after #10", $time);
    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
    join_any
        $display("@%0t: after join_any", $time);
        #80 $display("@%0t: finish after #80", $time);
end
```

(a) Code

```
@0: start fork...join_any example
@10: sequential after #10
@10: parallel start
@10: after join_any
@20: parallel after #10
@40: sequential after #30
@50: sequential after #10
@60: parallel after #50
@90: finish after #80
```

(b) Output



Creating Threads in A Class

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
class Gen_drive;

// Transactor that creates N packets
task run(input int n);
    Packet p;

    fork
        repeat (n) begin
            p = new();
            `SV RAND_CHECK(p.randomize());
            transmit(p);
        end
    join_none // Use fork-join_none so run() does not block
endtask

task transmit(input Packet p);
    ...
endtask
endclass

Gen_drive gen;

initial begin
    gen = new();
    gen.run(10);
    // Start the checker, monitor, and other threads
    ...
end
```

(a) Code



Dynamic Threads

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
program automatic test(bus_ifc.TB bus);
// Code for interface not shown
task check_trans(input Transaction tr);
    fork
        begin
            wait (bus.cb.data == tr.data);
            $display("@%0t: data match %d", $time, tr.data);
        end
    join_none // Spawn thread, don't block
endtask

Transaction tr;

initial begin
    repeat (10) begin
        tr = new(); // Create a random transaction
        `SV RAND_CHECK(tr.randomize());
        transmit(tr); // Send transaction into the DUT
        check_trans(tr); // Wait for reply from DUT
    end
    #100; // Wait for final transaction to complete
end
endprogram
```

(a) Code



Edge Triggered

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
event e1, e2;
initial begin
    $display("@%0t: 1: before trigger", $time);
    -> e1;
    @e2;
    $display("@%0t: 1: after trigger", $time);
end

initial begin
    $display("@%0t: 2: before trigger", $time);
    -> e2;
    @e1;
    $display("@%0t: 2: after trigger", $time);
end
```

(a) Code



Edge Triggered

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
event e1, e2;
initial begin
    $display("@%0t: 1: before trigger", $time);
    -> e1;
    @e2;
    $display("@%0t: 1: after trigger", $time);
end

initial begin
    $display("@%0t: 2: before trigger", $time);
    -> e2;
    @e1;
    $display("@%0t: 2: after trigger", $time);
end
```

(a) Code

```
@0: 1: before trigger
@0: 2: before trigger
@0: 1: after trigger
```

(b) Output



Level Sensitive

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
event e1, e2;

initial begin
    $display("@%0t: 1: before trigger", $time);
    -> e1;
    wait (e2.triggered);
    $display("@%0t: 1: after trigger", $time);
end

initial begin
    $display("@%0t: 2: before trigger", $time);
    -> e2;
    wait (e1.triggered);
    $display("@%0t: 2: after trigger", $time);
end
```

(a) Code



Level Sensitive

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
event e1, e2;

initial begin
    $display("@%0t: 1: before trigger", $time);
    -> e1;
    wait (e2.triggered);
    $display("@%0t: 1: after trigger", $time);
end

initial begin
    $display("@%0t: 2: before trigger", $time);
    -> e2;
    wait (e1.triggered);
    $display("@%0t: 2: after trigger", $time);
end
```

(a) Code

```
@0: 1: before trigger
@0: 2: before trigger
@0: 1: after trigger
@0: 2: after trigger
```

(b) Output



Mailbox

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

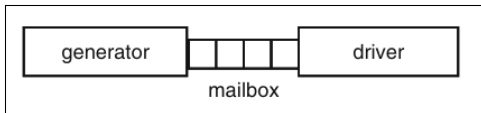
Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication





Unsynchronized Thread Communication

Threads and Interprocess Communication

Rehan Naeem

Fork Join Concept

Creating Threads in A Class

Dynamic Threads

Events and Mailbox

Unsynchronized Thread Communication

Synchronized Thread Communication

```
program automatic unsynchronized;

mailbox #(int) mbx;

class Producer;
task run();
    for (int i=1; i<4; i++) begin
        $display("Producer: before put(%0d)", i);
        mbx.put(i);
    end
endtask
endclass

class Consumer;
task run();
    int i;
    repeat (3) begin
        mbx.get(i);    // Get integer from mbx
        $display("Consumer: after get(%0d)", i);
    end
endtask
endclass

Producer p;
Consumer c;

initial begin
    // Construct mailbox, producer, consumer
    mbx = new();    // Unbounded
    p = new();
    c = new();

    // Run the producer and consumer in parallel
    fork
        p.run();
        c.run();
    join
end
endprogram
```

(a) Code



Unsynchronized Thread Communication

Threads and Interprocess Communication

Rehan Naeem

Fork Join Concept

Creating Threads in A Class

Dynamic Threads

Events and Mailbox

Unsynchronized Thread Communication

Synchronized Thread Communication

```
program automatic unsynchronized;

mailbox #(int) mbx;

class Producer;
task run();
    for (int i=1; i<4; i++) begin
        $display("Producer: before put(%0d)", i);
        mbx.put(i);
    end
endtask
endclass

class Consumer;
task run();
    int i;
    repeat (3) begin
        mbx.get(i);    // Get integer from mbx
        $display("Consumer: after get(%0d)", i);
    end
endtask
endclass

Producer p;
Consumer c;

initial begin
    // Construct mailbox, producer, consumer
    mbx = new();    // Unbounded
    p = new();
    c = new();

    // Run the producer and consumer in parallel
    fork
        p.run();
        c.run();
    join
end
endprogram
```

(a) Code

```
Producer: before put(1)
Producer: before put(2)
Producer: before put(3)
Consumer: after get(1)
Consumer: after get(2)
Consumer: after get(3)
```

(b) Output



Synchronized Thread Communication using Mailbox and Event

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Communication

Synchronized
Thread Communication

```
program automatic mbx_evt;
  mailbox #(int) mbx;
  event handshake;

  class Producer;
    task run();
      for (int i=1; i<4; i++) begin
        $display("Producer: before put(%0d)", i);
        mbx.put(i);
        @handshake;
        $display("Producer: after  put(%0d)", i);
      end
    endtask
  endclass : Producer

  class Consumer;
    task run();
      int i;
      repeat (3) begin
        mbx.get(i);
        $display("Consumer: after  get(%0d)", i);
        ->handshake;
      end
    endtask
  endclass : Consumer
```

(a) Code



Synchronized Thread Communication using Mailbox and Event

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

```
program automatic mbx_evt;
mailbox #(int) mbx;
event handshake;

class Producer;
task run();
    for (int i=1; i<4; i++) begin
        $display("Producer: before put(%0d)", i);
        mbx.put(i);
        @handshake;
        $display("Producer: after  put(%0d)", i);
    end
endtask
endclass : Producer

class Consumer;
task run();
    int i;
    repeat (3) begin
        mbx.get(i);
        $display("Consumer: after  get(%0d)", i);
        ->handshake;
    end
endtask
endclass : Consumer
```

(a) Code

```
Producer p;
Consumer c;

initial begin
    p = new();
    c = new();
    mbx = new();

    // Run the producer and consumer in parallel
    fork
        p.run();
        c.run();
    join
end
endprogram
```

(b) Code - Continued...



Synchronized Thread Communication using Mailbox and Event

Threads and
Interprocess
Communication

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Communication

Synchronized
Thread Communication

```
Producer: before put(1)
Consumer: after get(1)
Producer: after put(1)
Producer: before put(2)
Consumer: after get(2)
Producer: after put(2)
Producer: before put(3)
Consumer: after get(3)
Producer: after put(3)
```

(a) Output



Synchronized Thread Communication using Mailbox and Event

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication

remark

hello



References

Threads and
Interprocess
Communica-
tion

Rehan Naeem

Fork Join
Concept

Creating
Threads in A
Class

Dynamic
Threads

Events and
Mailbox

Unsynchronized
Thread Com-
munication

Synchronized
Thread Com-
munication



D. U. U. Fayyaz, "System verilog for verification (slides).".



Spears, *System Verilog for Verification*.